

Introduction to Image Analysis

Compulsory Assignment 1

Preliminary Version 0.1

Allan Rasmusson

2026.09.11

1 Assignment Description

This document (12 pages in total) describes a *preliminary* version the first compulsory assignment for the course “Introduction to Image Analysis” taking place at Faculty of Mathematics and Informatics, Vilnius University, Lithuania, Autumn 2026. This is the first of a total of three assignments; it will be scored between 0 and 100 points and weighted by 30% in the overall grade for the course.

In short, the assignment concerns the practical exercises of the first five weeks of the course, and, as such, can be interpreted as giving a summary of the material covered in the course so far and the practical implementation of it.

Engineering and Computer Science students must use C++ as implementation language while Data Scientists, System Biologists and pure Mathematics students are allowed to choose between C++ and Python. The choice of implementation language used in Compulsory Assignment 1 must be used for upcoming compulsory assignments 2 and 3 also. See section 1.1 for specific restrictions and which implementation libraries are allowed.

Although the assignment covers the practical exercises already handed out, some clarifications and minor adjustments are described in the following exercises. Also listed is the data handed out for each topic, but you may use any image you feel is best for your examples. However, some exercises will use designated images, which are either from the handouts or available in a zip file accompanying this document.

The student must include a written report with explanations of the background material, explained for an audience assumed not already familiar with it. Additionally, it must describe how the material was implemented including any choices made and how special cases were handled. The student may freely reorder the topics, exercises and descriptions of implementations to optimize the understanding of the material as a whole.

Besides the report, the implemented code (C++ or Python) must *also* be handed in. The report must include descriptions of how each example can be run on a command line in Ubuntu/Fedora Linux. For C++ code you must also describe how to compile it using a combination of `make` and `qmake`. See Appendix A for details.

The report must be submitted in pdf and uploaded together with the code as a single zip-file in the VU Emokymai system.

Deadline will be around the 23rd of October, but will be decided later. Also, a few changes may come to each assignment topic, but they will be finalized in the week the topic is covered in the lectures.

1.1 Code Limitations

1.1.1 C++

A code skeleton `CppSkeletonHist.zip` was handed out in the course (Week 3).

You are allowed to use basic standard libraries like `iostream`, `string`, and `cstdint` and `limits` for numerical types and `cmath` for rounding. `std::vector` is allowed for small dynamic lists, but *not* for image pixel data.

For linear algebra, use the Eigen library by including `Eigen/Core` and `Eigen/LU`.

Compile the image processing implementations as a stand-alone library, which is then used in (linked with) one or multiple example files. Use the handed out image viewer based on Qt for image viewing.

You may use the compilation process in the code hand-out which is described in the slides from Week 2.

1.1.2 Python

For Python implementations you are only allowed to use the following libraries:

```
from libtiff import TIFF
from matplotlib import pyplot as plt
import numpy as np
import math
import sys
```

with `numpy.matmul` and `numpy.linalg` used for matrix/vector calculations. Optionally you may use the `argparse` library to create and parse command line args; it's available via the usual Python and/or Linux package managers.

Image viewing is restricted to 8bit images as follows:

```
plt.imshow(aGrayScaleImage, cmap='gray', interpolation='nearest', vmin=0, vmax=255)
and
plt.imshow(someColorImage, interpolation='nearest', vmin=0, vmax=255)
```

Some Advice:

- Read the assignment description *at least* twice!
- Ask questions as early as possible.
- If you are having problems with some parts of the exercises, continue to the next and make it work with what you've got.
- Make sure you have access to the handed out files.
- Think about which part of the material is relevant for each weekly topic and include this in the report.

2 Assignment Details

2.1 Image Loading

Purpose: Get practical understanding of different image formats, their memory layout and how to combine/manipulate these. Visualizing the images is done as either RGB or gray-scale, but always using 8 bits per channel.

Implementations and Tasks:

- Image investigations:
Use the commandline tool `tiffinfo` to investigate the handed out images and describe the information you encounter. Comment on any observations you make.
- Image loading using `libtiff`
 - Loading images.
Load and visualize complete images - for C++ load tiled and striped formats.
`Kidney1.tif`
`Region_001_FOV_00041_Acridine_Or_Gray.tif`
`Region_001_FOV_00041_DAPI_Gray.tif`
`Region_001_FOV_00041_FITC_Gray.tif`
`SkinOverview.tif`
`TMA2-V2.tif`
If you encounter any anomalies investigate potential reasons for this using the information you extracted with the `tiffinfo` command. Describe the anomalies and reasons you find for them (you do *not* need to correct any anomalies as long the image is otherwise loaded correctly).
 - Load and visualize different sub-images of the sv5 file: `Kidney2_RGB2_20x.svs`.
Describe the internal tiff file structure and comment on why it could have been designed like this.
 - Loading subregions: Load and visualize subregions of images. If the image is tiled, read only the necessary individual tiles and combine these into a complete image of the subregion.
- Image combination:
 - Combine the three fluorescence images list below into one contiguous RGB image and save it to disk.
`Region_001_FOV_00041_Acridine_Or_Gray.tif`
`Region_001_FOV_00041_DAPI_Gray.tif`
`Region_001_FOV_00041_FITC_Gray.tif`

Data handout: Images in `ImgSet1.zip`

2.2 Intensity Transformations

Purpose: Gain practical experience with:

- Image intensity histograms.
- Image intensity transformations and how they affect the image by looking at the intensity histograms.
- Precision for intermediate calculations.

Implementations and Tasks: Only 8 bit grayscale images will be used, and so it's natural to use an **unsigned char** or equivalent 8-bit type to hold pixel intensities. You must describe your considerations regarding use of a different number format, e.g. **floats**, for intermediate calculations and any additional steps needed to conform with visualization of 8 bit images.

- Implement the power law for intensities (you may disregard the constant c)
Describe your implementation.
Visualize the effect of different values of the γ parameter.
(Hint: For some value of γ describe what results you expect (and why) and then explain if your implementation does output this or what could be the short-coming if it does not.)
- Implement piece-wise linear intensity transformations.
Give an example (and visualize the results of applying) one histogram stretching transformation and one thresholding.
- Implement histogram calculation for grayscale images.
- Implement histogram equalization.
Compare the piece-wise linear transformation applied above to the histogram equalization function.
Explain how/if the number of intensity values changes when performing a histogram equalization.
- Implement lookup table intensity transformations and use it to perform the intensity transformations above.
Discuss the differences between evaluating the intensity transform for each pixel and using a look-up table.
Are there any image/pixel formats that are not well suited to application of a look-up table?

Data handout: Images in `ImgSet2.zip`

2.3 Spatial Filtering and Multistep Image Processing

Purpose: Gain practical experience with:

- Image filtering in the spatial domain.
- Multi-step image processing.

Implementations and Tasks: Only 8 bit grayscale images will be loaded and visualized. Intermediate results will be calculated using *images* that holds pixel intensities in *floats*. Some form of conversion back to the original 8 bit image format is needed as viewing of only 8 bits images will be allowed.

- Implement separate separate image arrays (or image classes) for pixel intensities stored in **8-bit/unsigned char** and **float**, and methods to convert between the two formats. The conversion steps must be described in the report.
- Illustrate working image blurring by spatial filtering.
- Explain how linear filters can be used to implement numerical approximations of (partial) derivative operators. Give examples of this and of (magnitude of) gradient and the Laplacian.
- Illustrate image sharpening by unsharp mask and by using an intermediate Laplacian calculation. Comment on benefits/drawbacks between the two approaches.
- Redo example 3-43 using multistep image processing. Describe handling of intermediate results.

Data handout: Images in `ImgSet3.zip`, which include example 3-43.

2.4 Image Transformations

Purpose: Gain practical experience with:

- Different image coordinate system (both 2D world and local coordinates, 2D pixels indices).
- Composite transformations for transforming between image spaces and to apply arbitrary affine transformations.
- Application of affine transformations to 2D images.
- Image interpolation schemes for constructing result of affine image transformations.

Implementations and Tasks: Explain, implement and give examples of the following:

- Assigning coordinate systems to images and calculation of index-to-local and local-to-world transformations taking pixel size into account.
- Composite affine transformations.
- Nearest Neighbor and Bi-linear Interpolation for visualization of transformed images.
- Visualize different affine transformations using any appropriate image.
Use both nearest and bilinear interpolation types.
- Use the above for quality assurance of AI classification of kidney histology images as described on the following page.

Data: Images handed out in `ImgSet4.zip` and the images in the accompanying `BXX.zip` file.

Figure 1 shows a whole-slide-image (WSI) of a kidney biopsy stained by immunohistochemistry and imaged at 20x magnification. Inside a semi-manually defined region of interest (ROI) shown in yellow, with bounding box denoted “AI ROI”, AI classifiers were run in order to segment tissue compartments. These are depicted on top of the WSI as Glomeruli overlaid in green, Tubules (blue) and peritubular capillaries (PTC) in yellow.

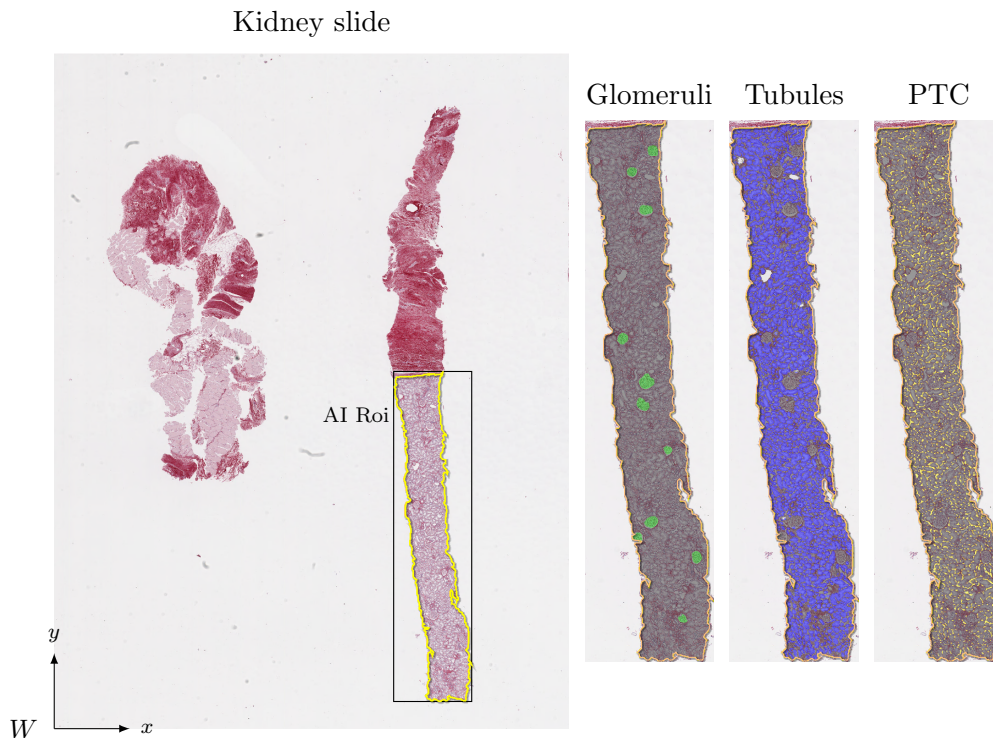


Figure 1: Kidney slide overview with Glomeruli, Tubules and peritubular capillaries (PTC) segmented in the “AI Roi” subregion.

The reason for training separate AI classifiers was to exploit the possibility to use different pixel sizes for the AI training patches independently of other tissue types. However, a price is that a pixel may be assigned *multiple* tissue classes. A quality assurance measurement is then the overlap of different tissues classes, which should ideally be 0. Calculating this *per glomeruli* will allow investigating if the glomeri itself has impact on the AI classifiers, but it is a separate task not considered here.

So, you are asked to calculate how much the Tubules and PTC overlaps with the Glomeruli given in Table 1. For brevity, you may consider only Glomeruli with id’s 3, 4 and 5. The overlap must be reported as area in both number of pixels and μm . Hint: First create and visualize overlays of the glomeruli regions on top of the WSI for each tissue class. *Then* calculate the overlaps. Also, calculate the overlaps in the grid/resolution/pixel size of the acquired WSI.

Additional: The “svs”-format used for the WSI, doesn’t use the `TIFF_TAGS` to store all information, but instead stores the data as text in the `IMAGE_DESCRIPTION` tag. *You are not asked to parse this information.* Instead all necessary information not available in the tags is given in table 2.

Table 1: ROI definitions: corners of each ROI in units of World coordinates W in Figure 2.4.1 (μm). ROIs for all segmented glomeruli are included for completeness; you only need to handle roi's 3,4 and 5 marked with *.

ROI	xMin	yMin	xMax	yMax
AI ROI	7784.5656	1113.5616	9547.7048	8631.5832
Glom 0	8634.5448	8131.5664	8782.6248	8282.6080
Glom 1	8349.7376	7847.7464	8492.3880	7981.5120
Glom 2	8524.4720	7315.6456	8710.5592	7456.8152
Glom 3*	8219.4272	5512.5248	8370.4688	5676.4000
Glom 4*	8516.5744	4893.5504	8733.7584	5108.7600
Glom 5*	8483.5032	4587.5184	8663.6672	4772.6184
Glom 6	8857.6520	3980.3904	8979.5712	4099.3480
Glom 7	8591.6016	2966.5360	8784.5992	3129.4240
Glom 8	8421.8032	2783.4104	8564.4536	2903.3552
Glom 9	9253.5192	2471.4552	9395.6760	2645.6960
Glom 10	9058.5472	1645.6624	9188.3640	1765.6072

Table 2: Image Attributes: For each filename **MPP** indicates either the acquisition size in $\mu\text{m px}^{-1}$ or if it can be read from the tiff tags. **Pixel intensities** indicate the (8-bit) intensity value for each tissue/pixel class in the classifier.

Filename	MPP	Region	Pixel intensities
BXX_Annon.svs	0.4936	World	Kidney tissue (RGB)
BXX_GLOM.tif	in tags	AI ROI	NB: <code>Scl_Glomeruli</code> trained by classifier, but no pixels identified in example. <label id="0" name="Background"/> <label id="1" name="Scl_Glomeruli"/> <label id="3" name="Glomeruli"/> <label id="255" name="Ignored"/>
BXX_TUB.tif	in tags	AI ROI	<label id="0" name="Background"/> <label id="1" name="Tubules"/> <label id="2" name="Glass"/> <label id="255" name="Ignored"/>
BXX_PTC.tif	in tags	AI ROI	<label id="1" name="Background"/> <label id="3" name="Capillaries"/> <label id="255" name="Ignored"/>

A Handing in code

A.1 Python

- You must hand in your code and report in a single zip-file.
- The archive will be extracted to a directory, e.g. **student-xx**.
- All handed out image data will be copied here in a directory named “data”.
- The code will then be run by:
passing a single file to the interpreter on the commandline
- For different examples, you may:
 - have different files, e.g. `imageIO.py`, `intensitytransform.py`, etc.
 - You may use different input args to run separate examples.
- You *must* describe in the report how to run each example.
 - **NB:** Do *not* rely on reviewers editing your code to run different examples!
Editing your code may introduce unintended behaviors and will *not* be done.
You risk not being scored for such examples.
- The python process could look like this on the Ubuntu command line:

```
> mkdir student7
> unzip student7.zip student7
> cd student7
> cd ‘‘CodeSubdirectory’’ #... or similar, describe in report
> cp -r /from/somewhere/data .
> python3 imagefilters.py ex1
> python3 imagefilters.py ex2
...
> python3 imagetransforms.py ex3 #... or similar, describe in report
...
```

A.2 C++

- Students using C++ will also hand in code and report in a single zip-file.
- The archive will be extracted to a directory, e.g. `student-xx`.
- All image data will be copied here in a directory named “data”.
- The code will then be compiled by running `make` and `qmake`.
- For different examples, you may:
 - have different files, e.g. `imageIO.cpp`, `intensitytransform.cpp`, etc.
 - You may use different input args to run separate examples.
- You *must* describe in the report how to run each example.
 - **NB:** Do *not* rely on reviewers editing your code to run different examples!
Editing your code may introduce unintended behaviors and will *not* be done.
You risk not being scored for such examples.
- The c++ process could look like this on the Ubuntu command line:

```
> mkdir student3
> unzip student3.zip student3
> cd student3
> cd ‘‘CodeSubdirectory’’ #... or similar, describe in report
> cp -r /from/somewhere/data .
> cd imagelib; make
> cd ../viewer; qmake
> export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:../imagelib
...
> ./imageIO ex1          # ... or similar, describe in report
...
```